

FORTIS-Akademie

An der Wiesenmühle 1

09224 Chemnitz

Fächerverbindender Unterricht

in den Fächern

Chemie, Physik, Biologie,

Gesundheit/Soziales

und

Informatiksysteme

Thema:

Hautkrebs – Entstehung, Risikofaktoren, Präventionsmöglichkeiten und die
Wiedereingliederung in den Alltag

Von

Eneko Reichert Vicario,

Lennard Pramhas,

Klassenstufe

BGY-24_3

Ort, Datum: Chemnitz/Grüna, 21.05.2026

Inhalt

1 Einleitung.....	2
2 Architektur und Technische Grundlagen	2
2.1 Das Model-View-Controller-Prinzip (MVC)	2
2.2 Softwarearchitektur	3
3 Datenmodell	4
3.1 Patientendaten (Patient.java)	4
3.2 Screeningdaten (ScreeningData.java)	5
4 Plausibilitätsprüfung (Validator.java)	6
4.1 Allgemeine Hilfsmethoden	6
4.2 Einfache Wertebereichsprüfungen	6
4.3 Komplexe Validierungsregeln (Screening)	7
5 Benutzeroberfläche und Anwendungssteuerung (MainController.java)	8
5.1 Struktur der Benutzeroberfläche	8
5.2 Interaktionslogik.....	9
5.3 Filter, Sortier und Bearbeitungsfunktionen	9
6 Persistenz: Dateioperator (FileHandler.java)	9
6.1 Export	9
6.2 Import.....	10
7 Optional: PDF-Funktion (ReportGenerator.java)	10
8 Teststrategie und Qualitätsassurance	10
8.1 Testgetriebene Entwicklung (TDD) in der eHKS-Anwendung	11
Vorteile für das Projekt.....	13
9 Zusammenfassung der Anforderungserfüllung	13
10 Schlussbemerkung	14
Anlagen: Quellenverzeichnis	14

1 Einleitung

Das Projekt Elektronisches Hautkrebsscreening (kurz eHKS) ist eine desktopbasierte Java-Anwendung zur strukturierten Erfassung, Validierung und Speicherung von Daten.

Der Hauptzweck der Software besteht darin, sämtliche für das Hautkrebsscreening relevanten Daten sowohl die allgemeinen Patientenstammdaten als auch die spezifischen Befunde und Untersuchungsinformationen vollständig, plausibel und datenschutzkonform zu erfassen. Diese Datensätze werden in CSV-Dateien¹ gespeichert, können jederzeit importiert und zur weiteren Verarbeitung exportiert werden. Daneben enthält die Anwendung eine integrierte, medizinisch fundierte Plausibilitätsprüfung, die sicherstellt, dass nur korrekte und vollständige Daten weiterverarbeitet werden.

2 Architektur und Technische Grundlagen

Das Projekt folgt einer klaren objektorientierten Struktur, die auf einer bewährten Trennung von Datenmodell, Geschäftslogik und Benutzeroberfläche basiert. Die Aufteilung gewährleistet eine hohe Modularität, verbessert die Testbarkeit und macht den Code wartbar und erweiterbar. Die Implementierung orientiert sich formal an den Prinzipien des Model-View-Controller-Entwurfsmusters (MVC)², ist jedoch in der Ausprägung durch die Besonderheiten von JavaFX³ geprägt und daher als angewandtes MVC-Muster zu betrachten.

2.1 Das Model-View-Controller-Prinzip (MVC)

Das Model-View-Controller-Muster ist eines der am weitesten verbreiteten Entwurfsmuster in der Softwarearchitektur. Es dient zur Strukturierung von Anwendungen, indem es die Anwendung in drei voneinander weitgehend unabhängige Bereiche unterteilt:

Model (Datenmodell)

Enthält die Arbeitsdaten des Programms (z. B. Nutzereingaben, Datenbankinhalte) sowie die zugehörige Geschäftslogik. Das Modell kennt keine Referenzen auf View oder Controller. Es ist rein datenorientiert und repräsentiert den Zustand der Anwendung. In unserem Projekt entsprechen die Klassen Patient und ScreeningData dem Model. Sie speichern die Daten, bieten Methoden zum Zugriff (getter/setter) und definieren keine GUI-Abhängigkeiten.

View (Präsentation)

¹ java.soeinding.de (keine Angabe): Java CSV Datei verarbeiten. URL: <https://java.soeinding.de/content.php/csvVerarbeitung> [letzter Zugriff am 20.05.2026]

² AxxG Blog (2013): Model-View-Controller (MVC) mit JavaFX. URL: <https://blog.axxg.de/model-view-controller-mit-javafx/> [letzter Zugriff am 20.05.2026]

³ AxxG Blog (2013): JavaFX: Controller, Events und UI-Elemente mit FXML. URL: <https://blog.axxg.de/javafx-controller-events-elemente-fxml/> [letzter Zugriff am 20.05.2026]

Die graphische oder textuelle Darstellung der Daten. Sie ist für die visuelle Ausgabe und die Empfangsstrukturen für Benutzereingaben verantwortlich, nicht für deren Verarbeitung. Die View enthält weder Daten noch Geschäftslogik, sondern besitzt lediglich Schnittstellen, um von der Controller-Schicht Benachrichtigungen oder Aktualisierungen zu erhalten. In der eHKS-Anwendung wird die View vollständig von der Klasse MainController über JavaFX-Komponenten⁴ (TextField, DatePicker, TableView, TabPane) generiert.

Controller (Steuerung)

Die vermittelnde Schicht, die für die Interaktion zwischen View und Model zuständig ist. Sie empfängt Benutzereingaben aus der View, ruft Validierungen (Validator) auf, aktualisiert das Model und steuert ggf. den Datenexport/Import (FileHandler). Der Controller kennt sowohl die View (um Werte zu lesen oder UI-Elemente zu aktualisieren) als auch das Model (um Änderungen vorzunehmen), ohne enge Kopplung an konkrete Implementierungsdetails. In der Anwendung ist dies die gesamte Klasse MainController.

Das Ziel des MVC-Musters ist ein flexibler Programmentwurf, der spätere Änderungen oder Erweiterungen erleichtert und die Wiederverwendbarkeit der einzelnen Komponenten ermöglicht. So können Änderungen an der Benutzeroberfläche (z. B. ein neues Layout in JavaFX) vorgenommen werden, ohne das Datenmodell (Patient, ScreeningData) berühren zu müssen. Ebenso können neue Validierungsregeln im Controller eingeführt werden, ohne die ViewStruktur zu beeinträchtigen.

Obwohl die vollständige Umsetzung des MVC-Musters oft ein Observer-Muster (z. B. ObservableList) zur automatischen Synchronisation zwischen Modell und View verwendet, ist die klare Trennung der Verantwortlichkeiten in unserem Projekt erkennbar:

- Patient.java / ScreeningData.java → Model
- MainController.java (GUI-Erstellung + Event-Handler) → View + Controller
- Validator.java / DataStorage.java / FileHandler.java → Controller-Erweiterung

2.2 Softwarearchitektur

Die Kernkomponenten der Anwendung sind wie folgt strukturiert:

Datenmodelle (Paket com.mycompany.ehks)

Die Klassen Patient und ScreeningData repräsentieren die zu speichernden Datenstrukturen. Sie enthalten ausschließlich Attribute sowie zugehörige Getter, Setter und Hilfsmethoden wie

⁴ AxxG Blog (2013): JavaFX: Eigene Klasse/Komponente in FXML nutzen. URL: <https://blog.axxg.de/javafx-custom-control-fxml/> [letzter Zugriff am 20.05.2026]

toString()) zur CSV konformen Serialisierung. Diese Klassen bilden den zentralen Teil des Model.

Validierungsmodul

Die Klasse Validator ist für sämtliche Plausibilitätsprüfungen zuständig. Sie enthält sowohl einfache, wertebasierte Prüfungen (etwa für Geschlecht oder Versichertenart) als auch komplexe, kontextbezogene Regeln, die mehrere Felder miteinander verknüpfen. Die Logik befindet sich somit vollständig im Controller-Bereich, nicht im Model.

Persistenzschicht

Die Klassen DataStorage und FileHandler koordinieren den Speicher und Ladevorgang der Daten. Während DataStorage die aktuell im Arbeitsspeicher befindlichen Datensätze als beobachtbare Listen (ObservableList) verwaltet, übernimmt FileHandler die konkrete CSVImport und Exportlogik. DataStorage fungiert dabei als zentraler Zugriffspunkt für alle Komponenten und ermöglicht es, den Datenbestand über ein Singleton-Muster konsistent zu verwalten.

Steuerungs und UI-Schicht

Die Klasse MainController verwaltet sämtliche GUI-Elemente, Ereignishandler und Benutzerinteraktionen. Sie greift auf die beiden vorigen Schichten zu, um Eingaben zu validieren, Daten zu speichern oder zu bearbeiten. Die Anwendung selbst wird über MainApp initialisiert, das die JavaFX-Stage instanziiert und die Benutzeroberfläche anzeigt.

Die Trennung zwischen den Schichten ist deutlich ausgeprägt: Das Model enthält keine GUI-Logik, die View enthält keine Validierungs oder Persistenzfunktionen, und der Controller vermittelt zwischen beiden, ohne seine eigenständige Verantwortung aufzugeben.

3 Datenmodell

3.1 Patientendaten (Patient.java)

Die Klasse Patient repräsentiert den Versicherten und seine zugehörigen Personen sowie Praxisdaten. Alle Attribute sind als private deklariert, ein Standardkonstruktor und die üblichen Getter und Setter-Methoden ermöglichen die vollständige Datenkapselung. Folgende Felder sind vorgesehen:

- Persönliche Daten: Name, Vorname und Geburtsdatum (gespeichert als LocalDate)
- Versicherungsdaten: VersichertenID, Versichertenart (1 für Mitglied, 3 für Familienangehöriger, 5 für Rentner) und Geschlecht (M, F, UN)
- Adresse: Straße, Hausnummer, PLZ und Ort

- Praxisdaten: Betriebsstättennummer, Lebenslange Arztnummer und Arztname

Die Methode `isValid()` prüft, ob alle erforderlichen Felder vollständig und korrekt ausgefüllt sind, und delegiert diese Prüfung an die zentrale Validierungsklasse `Validator`.

Zusätzlich sind die Methoden `equals()` und `hashCode()` so angepasst, dass Patienten eindeutig anhand ihrer VersichertenID identifiziert werden können. Dies ist entscheidend für die korrekte Verknüpfung von Patienten und zugehörigen Screening-Datensätzen. Die Methode `toString()` erzeugt eine semikolongetrennte Zeichenkette, die exakt dem Datei-Import-Format entspricht.

3.2 Screeningdaten (`ScreeningData.java`)

Die Klasse `ScreeningData` ist das zentrale Modell für alle Untersuchungsdaten. Sie ist nach dem KBV-Anforderungskatalog strukturiert und umfasst folgende Inhaltsgruppen:

Verdachtsdiagnose (1.1)

Die Angabe der Verdachtsdiagnose ND (Neudiagnose), sowie bei Vorliegen die Differenzierung zwischen malignem Melanom, Basalzellkarzinom, spinozellulärem Karzinom, anderem Hautkrebs und sonstigen dermatologisch abklärungsbedürftigen Befunden. Zudem ist festgehalten, ob der Patient an einen Dermatologen überwiesen wurde.

Überweisung und Kontext (2.1–2.2)

Hier werden Angaben zur Durchführung einer Gesundheitsuntersuchung (Checkup 35), zur Überweisung im Rahmen des Hautkrebsscreenings sowie zur Verdachtsdiagnose des überweisenden Arztes erfasst.

Befund des Dermatologen (2.3)

Die Verdachtsdiagnose des dermatologisch tätigen Arztes ist separat abgelegt, inklusive der gleichen Differenzierung wie bei der ND-Angabe.

Biopsie/Exzision (2.4)

Die Angabe, ob eine Entnahme von Gewebeproben durchgeführt wurde, die Anzahl der Biopsien sowie die Information, ob weitere Therapien oder Diagnostiken vorgenommen wurden.

Histopathologie (2.5)

Die abschließende diagnostische Sicherung durch mikroskopische Untersuchung. Je nach Befund sind zusätzliche Pflicht oder optionale Felder vorgesehen: Klassifikation und Breslow-Tumordicke bei Melanom, horizontaler und vertikaler Durchmesser bei Basalzellkarzinom sowie

Klassifikation und Grading bei spinözellulärem Karzinom. Darüber hinaus werden nichtmaligne Befunde wie atypische Nävi oder aktinische Keratosen erfasst.

Die Klasse enthält keine eigene Validierungsmethode, sondern greift auf `Validator.isValidScreening(ScreeningData)` zurück, um komplexe, mehrstufige Prüfungen durchzuführen.

Für die visuelle Orientierung im Benutzerinterface ist die Methode `getStatusColor()` implementiert, die auf Basis der vorliegenden Befunde Farbcodes zurückgibt. Diese dienen der schnellen Einschätzung des Untersuchungsstatus: Eine grüne Hintergrundfarbe deutet auf eine normale Gesundheitsuntersuchung ohne Auffälligkeiten hin, gelbe Hintergründe kennzeichnen diagnostische Verdachtsmomente, und rote Hintergründe signalisieren bereits durchgeführte Interventionen wie Biopsien.

Die Methode `toString()` wiederum erzeugt eine CSVkonforme Repräsentation des gesamten Screening-Datensatzes und umfasst alle relevanten Felder in der korrekten Reihenfolge für den Export.

4 Plausibilitätsprüfung (Validator.java)

Die Klasse `Validator` bildet den zentralen Bestandteil der Validierungslogik. Sie enthält ausschließlich statische Methoden und dient als zentrale Schnittstelle für sämtliche Plausibilitätsregeln. Die Implementierung orientiert sich exakt an den festgelegten KBVRegeln, wobei jeder Fehler in der Datenüberprüfung als aussagekräftige Fehlermeldung zurückgegeben wird.

4.1 Allgemeine Hilfsmethoden

Eine private Hilfsmethode `isBlank(String)` prüft, ob eine Zeichenkette null ist oder nur aus Leerzeichen besteht. Dies wird verwendet, um leere oder unvollständige Eingaben zu erkennen.

4.2 Einfache Wertebereichsprüfungen

- `isValidGeschlecht(String)` erlaubt ausschließlich die Werte M, F oder UN.
- `isValidVersichertenArt(String)` akzeptiert nur 1, 3 oder 5.
- `isValidAnzahlBiopsien(int)` beschränkt den Wertebereich auf ganze Zahlen zwischen 0 und 99.
- `isValidTumordurchmesser(Double)` fordert Dezimalwerte im Bereich von 0,1 bis 999,9 Millimeter.

Weitere Hilfsmethoden für Klassifikationen (Melanom, Spinozellulär), Grading und Breslow-Systematik prüfen, ob die angegebenen Zeichenfolgen Teil des jeweiligen erlaubten Wertebereichs sind.

4.3 Komplexe Validierungsregeln (Screening)

Die Methode `isValidPatient(...)` prüft zunächst, ob alle Pflichtfelder ausgefüllt sind. Anschließend wird geprüft, ob das Geburtsdatum das Format JJJJMMTT aufweist und ob es sich um ein gültiges Datum handelt.

Die Methode `getValidationErrors(ScreeningData)` ist die mächtigste Komponente: Sie iteriert durch sämtliche Screening-Felder und erzeugt eine Liste aller aufgetretenen Fehler, die als Textnachrichten zurückgegeben wird. Das Vorhandensein einer leeren Liste bedeutet, dass der Datensatz vollständig plausibel ist. Die Prüfroutine gliedert sich in mehrere logische Blöcke:

Verdachtsdiagnose (1.1)

Die Angabe Verdachtsdiagnose ND muss auf Ja gesetzt sein. Daneben ist streng darauf zu achten, dass nur eine einzige Diagnose aus den sechs möglichen Optionen (Melanom, BCC, SCC, anderer Hautkrebs, sonstiger Befund, Überweisung an Dermatologen) gewählt wird. Ein weiteres Verbot besteht darin, dass die Überweisung an einen Dermatologen nicht gleichzeitig mit einer anderen Diagnosenangabe erfolgen darf.

Überweisungskontext (2.1–2.2)

Es ist erforderlich, dass sowohl eine Gesundheitsuntersuchung als auch eine Überweisung im Rahmen des Hautkrebsscreenings erfolgt ist. Zudem muss der überweisende Arzt bestätigen, dass er selbst bereits eine Hautkrebsscreeninguntersuchung durchgeführt hat. Bei Angabe der Verdachtsdiagnose des überweisenden Arztes muss exakt eine Diagnose ausgewählt werden, bei Negativangabe darf keine Diagnose angegeben sein.

Dermatologenbefund (2.3)

Auch die Verdachtsdiagnose des Dermatologen muss auf Ja gesetzt sein. Wie bei der ND-Angabe ist hier ebenfalls eine einzige Diagnose zulässig.

Biopsie und Histopathologie (2.4–2.5)

Wenn keine Biopsie durchgeführt wurde, dürfen keine histopathologischen Befunde gemeldet werden. War hingegen eine Entnahme stattgefunden, so muss die Anzahl der Biopsien zwischen 0 und 99 liegen. Zudem muss mindestens eine HautkrebsDiagnose aus der Liste Melanom, BCC, SCC oder anderer Hautkrebs angegeben werden, sowie mindestens eine nichtmaligne Diagnose (Nävus, aktinische Keratose, sonstige Veränderung). Daneben gibt es

absolute Ausschlussregeln: anderweitige Therapie und keine weitere Therapie dürfen nicht gleichzeitig aktiv sein.

Histopathologische Pflichtfelder

Je nach Diagnose sind zusätzliche Felder zwingend erforderlich: Bei Melanom ist die Klassifikation (in situ oder invasiv) verpflichtend, bei Basalzellkarzinom der horizontale Tumordurchmesser im Wertebereich 0,1 bis 999,9, und bei spinözellulärem Karzinom wiederum die Klassifikation.

Patientenbezug

Die Screening-Daten müssen einem existierenden, validen Patienten zugeordnet sein. Existiert kein passender Patient oder ist dessen Datensatz nicht plausibel, so wird die Überprüfung mit einer Fehlermeldung abgebrochen.

5 Benutzeroberfläche und Anwendungssteuerung (MainController.java)

Die Klasse MainController ist für die gesamte Steuerung der grafischen Benutzeroberfläche verantwortlich. Sie initialisiert sämtliche Komponenten, bindet Event-Handler ein und koordiniert die Interaktion zwischen Benutzer, Datenmodell und Persistenzschicht.

5.1 Struktur der Benutzeroberfläche

Die Benutzeroberfläche ist in ein TabbedInterface mit drei Tabs strukturiert:

Tab Patienten: Hier werden alle Daten des Patienten erfasst. Eingabefelder umfassen Name, Vorname, Geburtsdatum (über einen DatePicker), VersichertenID, Adresse sowie Angaben zur Versichertenart und zum Geschlecht. Zusätzlich sind Felder für die Arztpraxis vorgesehen: Betriebsstättennummer, Arztnummer und Name der behandelnden Ärztin bzw. des behandelnden Arztes.

Tab Untersuchungen: Dieser Tab enthält das komplexe Formular zur Erfassung aller Screening-Daten. Die Felder sind thematisch gruppiert: Verdachtsdiagnosen, Überweisung, Befund des Dermatologen, Biopsie sowie Histopathologie. Gewisse Checkboxen steuern die Sichtbarkeit untergeordneter Eingabefelder: So werden beispielsweise die Eingabefelder für den horizontalen und vertikalen Tumordurchmesser nur dann eingeblendet, wenn das Melanom aktiviert ist.

Tab Daten und Export: In diesem Tab werden die erfassten Datensätze als Tabellen angezeigt. Über Filterfunktionen können Patienten anhand ihres Namens durchsucht werden. Zusätzlich

stehen hier die Buttons für Import, Export und das Löschen ausgewählter Datensätze zur Verfügung.

5.2 Interaktionslogik

Das Formular ist so aufgebaut, dass dynamische Abhängigkeiten zwischen den Eingabefeldern automatisch erkannt und umgesetzt werden. Sobald die Checkbox Verdachtsdiagnose ND ausgewählt wird, werden alle zugehörigen Diagnose-Checkboxes deaktiviert und zurückgesetzt. Eine Selektion von Biopsie/Exzision aktiviert das Eingabefeld für die Anzahl der entnommenen Proben.

Beim Speichern eines Datensatzes wird zunächst eine Plausibilitätsprüfung durchgeführt. Fehlt ein erforderliches Feld oder ist eine Eingabe in einem falschen Format, wird eine Warnmeldung eingeblendet, die den Benutzer auf die konkreten Fehler hinweist.

Fehlerhafte oder unvollständige Datensätze werden nicht in den Speicher geschrieben. Lediglich vollständig plausibel aufgebaute Einträge werden als neue oder aktualisierte Datensätze in der internen Liste geführt.

5.3 Filter, Sortier und Bearbeitungsfunktionen

Der Suchfilter im Tab Daten & Export ermöglicht die schnelle Lokalisierung von Patienten anhand ihres Nach oder Vornamens. Die Tabellen sind sortierbar; Klick auf eine Spaltenüberschrift sortiert die Liste auf oder absteigend. Ein Doppelklick auf eine Tabellenzeile öffnet den entsprechenden Bearbeitungsdialog und füllt die Eingabefelder mit den gespeicherten Daten.

Das Löschen eines Datensatzes erfolgt immer mit einer Sicherheitsabfrage. Beim Löschen eines Patienten werden automatisch alle ihm zugeordneten Screening-Datensätze mit entfernt, um Inkonsistenzen zu vermeiden.

6 Persistenz: Dateioperator (FileHandler.java)

Die Klasse FileHandler übernimmt die Vollständigkeit und Konsistenz der Daten beim Import und Export.

6.1 Export

Beim Export werden zwei CSV-Dateien erstellt: eine für die Patientendaten (..._patients.csv) und eine zweite für die Screening-Daten (..._screenings.csv). Die Dateinamen basieren auf dem vom Benutzer gewählten Basisnamen, ergänzt um die entsprechenden Suffixe.

Jede Datei beginnt mit einer Headerzeile, die die Feldnamen in der korrekten Reihenfolge enthält. Anschließend werden alle Datensätze mithilfe der toString()Methode der jeweiligen Klasse in CSV-Form serialisiert.

Vor dem Export wird eine finale Validierung aller Datensätze durchgeführt. Existiert mindestens ein Datensatz, dessen Plausibilität nicht gewährleistet ist, wird der Exportvorgang abgebrochen und eine entsprechende Warnung angezeigt. Dies verhindert das Speichern von fehlerhaften Daten in der Exportdatei.

6.2 Import

Der Import besteht aus zwei Schritten: Zunächst wird die Patientendatei geladen, anschließend die Screening-Datei. Beim Laden der Screening-Daten wird geprüft, ob der zugehörige Patient bereits in der internen Speicherliste existiert. Falls nicht, wird automatisch ein neuer Patient mit dem Wort Unbekannt als Name und der Versicherten-ID als eindeutigen Schlüssel angelegt.

Während des Imports werden Warnungen erfasst, die bei auftretenden Fehlern (etwa ungültige Datumsangaben oder nichtnumerische Werte in Zahlenfeldern) an den Benutzer zurückgemeldet werden. So kann der Benutzer erkennen, welche Datensätze möglicherweise unvollständig oder ungenau importiert wurden.

Beim Import werden alle zuvor gespeicherten Datensätze gelöscht, um einen konsistenten Initialzustand zu gewährleisten.

7 Optional: PDF-Funktion (ReportGenerator.java)

Die Klasse ReportGenerator ist für die Erstellung von PDF-Kurzberichten vorgesehen. Sie nutzt die iTextBibliothek, um einen strukturierten Text zu generieren, der Patientendaten, Untersuchungsergebnisse und eine Empfehlung enthält. Die Empfehlung richtet sich nach der Farbcodierung des Screening-Datensatzes und differenziert zwischen dringender Behandlung, regelmäßigen Kontrollen und Routineuntersuchung.

Derzeit ist diese Funktion in der Anwendung deaktiviert, da sie nicht Bestandteil der Pflichtenforderungen ist und die anderen optionalen Funktionen priorisiert wurden. Die Implementierung ist dennoch im Quellcode enthalten und kann bei Bedarf aktiviert werden.

8 Teststrategie und Qualitätsassurance

Zur Sicherstellung der Korrektheit der Validierungslogik wurde ein testgetriebener Ansatz verwendet, der auf der direkten Validierung von Typen und Werten basiert. Beispielsweise wurde für die Prüfung der Verdachtsdiagnosen ein Testfall definiert, bei dem versucht wurde, zwei verschiedene Diagnosen (etwa Melanom und Basalzellkarzinom) gleichzeitig zu aktivieren. Dieser Versuch musste von der Validierung korrekt als unzulässig zurückgewiesen werden.

In weiteren Testszenarien wurde sichergestellt, dass bei fehlender Biopsie keine Histopathologie-Felder gesetzt sein dürfen, und dass bei aktivierter Überweisung an einen Dermatologen keine weiteren Diagnosen angegeben sein dürfen. Jeder Testfall wurde separat formuliert, die Implementierung vorgenommen, und die Fehlermeldungen so angepasst, dass sie vom Anwender korrekt interpretiert und behoben werden konnten.

Die Testung hat dazu geführt, dass sämtliche KBVRegeln zuverlässig umgesetzt wurden. Die Validierung ist nun so gestaltet, dass sie aussagekräftige Fehlermeldungen erzeugt, die den Benutzer direkt auf die Ursache des Problems hinweisen.

8.1 Testgetriebene Entwicklung (TDD)⁵ in der eHKS-Anwendung

Zur Sicherstellung einer hohen Codequalität und der korrekten Implementierung der komplexen KBV-Plausibilitätsregeln wurde die Entwicklung der Anwendung nach den Prinzipien der testgetriebenen Entwicklung (Test-Driven Development, TDD)⁶ begleitet. TDD ist ein iterativer Entwicklungsansatz, bei dem Tests vor der eigentlichen Implementierung geschrieben werden. Dieser Ansatz ist Bestandteil der agilen Softwareentwicklung und wurde in den Kernprozess der Entwicklungsarbeit integriert, insbesondere für die Validierungskomponente Validator.java.

Grundlegender Ablauf nach TDD: Red–Green–Refactor

Die Entwicklung folgte dem bewährten Red–Green–Refactor-Zyklus, der aus drei Phasen besteht:

Red (Rot): Zunächst wird ein fehlgeschlagener (roter) Unit-Test⁷ geschrieben, der das gewünschte Verhalten exakt beschreibt, das noch nicht implementiert ist. Dieser Test formuliert die Anforderung in maschinenlesbarer Form.

Green (Grün): Anschließend wird der minimal notwendige Code implementiert, um den fehlgeschlagenen Test erfolgreich (grün) auszuführen. Der Fokus liegt hierbei auf Funktionalität, nicht auf Perfektion.

⁵ IBM (keine Angabe): Was ist Test-Driven Development (TDD)?. URL: <https://www.ibm.com/de-de/think/topics/test-driven-development> [letzter Zugriff am 20.05.2026]

⁶ Coursera Staff (2025): Test Driven Development verstehen. URL: <https://www.coursera.org/de-DE/articles/test-driven-development> [letzter Zugriff am 20.05.2026]

⁷ JUnit Team (keine Angabe): JUnit 5 User Guide. URL: <https://junit.org/junit5/docs/current/user-guide/> [letzter Zugriff am 20.05.2026]

Refactor (Refaktor): Sobald der Test grün ist, wird der Code verbessert (z. B. Duplikate entfernt, Namenskonventionen angepasst, Modularität erhöht), ohne die getestete Funktionalität zu verändern. Die bereits geschriebenen Tests dienen dabei als Sicherheitsnetz.

Dieser Zyklus wird für jede einzelne Funktion in der eHKS-Anwendung also für jede spezifische KBV-Plausibilitätsregel wiederholt.

Beispielhafte Anwendung an der Klasse Validator

Die Klasse Validator ist für die Validierung der Screening-Daten nach dem KBV-

Plausibilitätenkatalog verantwortlich. Die TDD-Vorgehensweise wird am Beispiel der Regel für die Verdachtsdiagnose (KBV-Abschnitt 1.1: „nur eine Angabe ist möglich“) wie folgt dargestellt:

Schritt Red: Der fehlgeschlagene Test

Zunächst wurde ein Testfall erstellt, der bewusst fehlschlägt. Ziel war es, sicherzustellen, dass die Validierung das Setzen von zwei Diagnosen (z. B. „Malignes Melanom“ und „Basalzellkarzinom“) als ungültig erkennt.

```
@Test
void shouldRejectMultipleDiagnoses() {
    ScreeningData screening = new ScreeningData();
    screening.setVerdachtsdiagnoseND(true);
    screening.setMalignesMelanom(true);
    screening.setBasalzellkarzinom(true); // Verstoß

    List<String> errors = Validator.getValidationErrors(screening);

    assertTrue(errors.stream()
        .anyMatch(msg -> msg.contains("nur eine")
            && msg.contains("1.1.2-1.1.7")),
        "Erwartete Fehlermeldung: 'nur eine Angabe'");
}
```

Zu diesem Zeitpunkt existierte die Methode getValidationErrors() noch nicht, der Test schlug also rot fehl. Das Schreiben des Tests vor der Implementierung zwang dazu, sich zuerst Gedanken über die Schnittstelle und das erwartete Verhalten zu machen.

Schritt Green: Minimaler, funktionierender Code

Anschließend wurde die Methode `getValidationErrors()` so implementiert, dass sie für diesen konkreten Fall die korrekte Fehlermeldung zurückgibt. Die Initialimplementierung war bewusst minimal sie zählte die aktiven Diagnosen und warf eine Exception, wenn mehr als eine vorhanden war. Komplexere Logiken wie die Ausschlussregel für die „Überweisung an Dermatologen“ wurden noch nicht berücksichtigt.

```
public static List<String> getValidationErrors(ScreeningData s) {
    List<String> errors = new ArrayList<>();
    if (!s.isVerdachtsdiagnoseND()) {
        errors.add("VerdachtsdiagnoseND muss 'Ja' sein.");
        return errors;
    }

    int anzahl = (s.isMalignesMelanom() ? 1 : 0) + (s.isBasalzellkarzinom() ? 1 : 0);
    if (anzahl > 1) {
        errors.add("1.1: Es darf nur eine Angabe in 1.1.2-1.1.7 gegeben sein.");
    }
    return errors;
}
```

Der Test lieferte nun grün. Andere, nicht implementierte Fälle (z. B. „Überweisung an Dermatologen“) waren bis dahin noch nicht relevant.

Schritt Refactor: Verbesserung und Erweiterung

In mehreren Iterationen wurde der Code schrittweise erweitert und optimiert:

Die Zähllogik wurde in eine eigene Hilfsmethode `countDiagnosen()` ausgelagert (DRY-Prinzip).

Die Ausschlussregel für „Überweisung an Dermatologen“ wurde als neuer Testfall (`shouldRejectDiagnoseWithDermatologistReferral`) geschrieben und dann implementiert.

Die Fehlermeldungen wurden so verbessert, dass sie die aktuelle Anzahl* der Diagnosen enthielten, um dem Anwender eine schnellere Fehlerbehebung zu ermöglichen.

Durch die kontinuierliche Ausführung der Tests nach jeder Änderung war sichergestellt, dass keine bestehende Funktionalität durch Refactorings beschädigt wurde.

Vorteile für das Projekt

Der Einsatz von TDD brachte drei zentrale Vorteile für das Projekt:

- **Klarheit der Anforderungen:** Durch das Schreiben des Tests vor der Implementierung wurde die KBV-Spezifikation „Verdachtsdiagnose: nur eine Angabe ist möglich“ in eine präzise, testbare Spezifikation übersetzt.
- **Vermeidung von Regressionen:** Die Sammlung von Unit-Tests (im Projekt als `ValidatorTest` strukturiert) dient als „ausführbare Dokumentation“. Jede zukünftige Änderung an der Validierungslogik wird sofort auf ihre Korrektheit geprüft.

- **Qualität des Codes:** TDD erzwingt lose Kopplung und klare Schnittstellen. Die Validator-Klasse ist reines Datenverarbeitungsmodul ohne GUI- oder Persistenzabhängigkeiten perfekt testbar.

9 Zusammenfassung der Anforderungserfüllung

Die Anwendung erfüllt alle gestellten Pflichtfunktionen:

- Die Anwendung ist in Java implementiert und über eine grafische Benutzeroberfläche bedienbar
- Patientenstammdaten können erfasst, gespeichert und wieder importiert werden
- Alle Screening-Daten können vollständig und plausibel eingegeben werden
- Die Implementierung erfolgte objektorientiert nach modernen Entwurfsmustern
- Eine Filterfunktion zur Eingrenzung nach Nachname ist implementiert
- Die Anwendung erlaubt die Farbmarkierung von Datensätzen nach ihrem Status

Die optionalen Funktionen zur PDF-Erstellung ist technisch realisiert, jedoch in der aktuellen Version deaktiviert.

10 Schlussbemerkung

Die eHKS-Anwendung ist eine vollständig funktionsfähige Lösung zur elektronischen Erfassung von Hautkrebsscreening-Daten. Die klare Trennung der Funktionalität in untereinander abgestimmte Module gewährleistet eine hohe Stabilität und Wartbarkeit des Quellcodes. Die starke Bindung an die offiziellen KBV-Vorgaben sichert die medizinische Plausibilität aller erfassten Datensätze und schließt fehlerhafte Eingaben frühzeitig aus.

Die Anwendung eignet sich somit nicht nur als Lernmedium für die in das Projekt eingebundenen Fächer, sondern auch als Grundlage für eine spätere Weiterentwicklung zu einem kommerziellen Praxisverwaltungssystem im Bereich der Dermatologie.

Anlagen: Quellenverzeichnis

AxxG Blog (2013): JavaFX: Controller, Events und UI-Elemente mit FXML. URL: <https://blog.axxg.de/javaafx-controller-events-elemente-fxml/> [letzter Zugriff am 20.05.2026]

AxxG Blog (2013): JavaFX: Eigene Klasse/Komponente in FXML nutzen. URL: <https://blog.axxg.de/javaafx-custom-control-fxml/> [letzter Zugriff am 20.05.2026]

AxxG Blog (2013): Model-View-Controller (MVC) mit JavaFX. URL: <https://blog.axxg.de/model-view-controller-mit-javafx/> [letzter Zugriff am 20.05.2026]

Coursera Staff (2025): Test Driven Development verstehen. URL: <https://www.coursera.org/de-DE/articles/test-driven-development> [letzter Zugriff am 20.05.2026]

IBM (keine Angabe): Was ist Test-Driven Development (TDD)?. URL: <https://www.ibm.com/de-de/think/topics/test-driven-development> [letzter Zugriff am 20.05.2026]

java.soeinding.de (keine Angabe): Java CSV Datei verarbeiten. URL: <https://java.soeinding.de/content.php/csvVerarbeitung> [letzter Zugriff am 20.05.2026]

JUnit Team (keine Angabe): JUnit 5 User Guide. URL: <https://junit.org/junit5/docs/current/user-guide/> [letzter Zugriff am 20.05.2026]